

Build Specification

Staffing Application Automation System

Central hub, three-level web portal, and consultant desktop application

Version 1.1 | July 4, 2026

Prepared for the business owner. To be handed to the development team.

Confidential — internal use only

Contents

(In Word: right-click the list above and choose Update Field to fill in page numbers.)

1. Purpose and Scope

This document describes a system that automates job applications for a staffing agency's bench consultants. Every 4 hours the system finds new job postings that match each consultant, tailors that consultant's resume for each job, and fills out the application forms on the job portals. A human — the consultant — reviews and clicks the final submit on every application.

The agency owner controls all data from one central point. Consultants run a small application on their personal machines that does only the browser work. Nothing sensitive lives permanently on consultant machines.

1.1 Decisions already made

The following decisions were made during design and are final unless the owner changes them:

- Every application goes out under the consultant's own name, email, and portal accounts. The system does the labor; the identity is always the consultant's.
- The final submit click is made by the consultant, never by the machine. The app fills the form and stops at the review screen.
- Consultants work independently. If one consultant's login expires, their queue waits for them. Jobs are never moved to another consultant.
- Resume tailoring may reword and reorder what is already true. It must never invent skills, tools, employers, dates, or accomplishments. This rule is enforced in the central system where consultants cannot change it.
- Answer approval flow: the consultant fills in an unknown answer, their recruiter approves it, then it becomes reusable. Answers about salary expectations and work authorization always route to the owner for approval.
- All records are kept indefinitely: every application, the exact resume file submitted, and every answer given on every form.
- For every matched job the system identifies the job poster and obtains their business email and phone. If the poster is missing or generic, it finds two active talent-acquisition contacts at that company in that work location. Contact data comes from licensed data providers, never from scraping LinkedIn directly.
- Contacts are stored centrally and reused for 90 days before a fresh pull is allowed, to minimize paid lookups. Consultants see the contact attached to each job in their own queue; only the owner and recruiters can search the full contact store.

2. System Overview

The system has three parts:

1. **Central Hub** — a cloud server owned by the agency. It searches for jobs, tailors resumes, assigns work queues, stores all records, and holds the answer bank. It runs the 4-hour cycle.
2. **Web Portal** — a website served by the hub with three access levels: Owner, Recruiter, and Consultant. Each level sees only what it is allowed to see.
3. **Consultant Desktop App** — a small program on each consultant's personal machine. It pulls that consultant's queue from the hub, opens the job portals in a real browser using the consultant's own saved logins, fills the forms, stops at the review screen, and reports everything back to the hub.

Data flows one way for sensitive material: the hub sends each consultant only the jobs, resume files, and approved answers needed for their current queue. The consultant app sends back status reports and form question-and-answer records. Consultant machines keep no permanent data.

3. Central Hub — Functional Specification

3.1 Job discovery engine (runs every 4 hours)

- Reads each consultant's search criteria file: job titles, keywords, locations, work type (contract, full-time, part-time, C2C/W2), minimum pay, excluded companies.
- Pulls new postings from sources that permit it cleanly: the public job feeds of Greenhouse, Lever, and Workable company career pages; job alert emails from Indeed and LinkedIn parsed by the system; and a paid search feed (such as SerpAPI for Google Jobs) for coverage.
- Applies a cheap keyword pre-filter first (title and location match) so only plausible postings go to the AI matching step. This cuts AI cost by roughly 90 percent.
- Removes duplicates using a company + title + location fingerprint stored in the database. Only postings first seen after the previous run move forward.
- For each surviving posting, an AI call decides match or no-match against each relevant consultant's criteria and classifies the work type. Matched jobs enter that consultant's queue.

3.2 Resume tailoring engine

- For each matched job, the hub sends the job description plus the consultant's base resume to the AI with a locked instruction set: keep the existing format, section order, and voice; reorder and reword bullet points to mirror the job description's keywords; surface relevant real experience.
- Hard rule, enforced in the hub's system prompt and not editable by any consultant or recruiter: the AI may rephrase and emphasize what is already true but must never fabricate. A second cheap AI check compares the tailored resume against the base resume and flags any claim that has no basis in the original. Flagged resumes go to the recruiter for review instead of the queue.

- Output is a PDF named `Company_Title_Date.pdf`, stored in the hub and linked to the application record. The consultant app downloads it per job; there is no bulk resume download anywhere in the system.

3.3 Queue assignment

- Each consultant has one queue. A queue item contains: the job link, the portal type, the tailored resume, and the approved answers likely needed for that portal's form.
- If one job matches several consultants, it appears in each of their queues — each applies under their own name. The dashboard flags the overlap for visibility.
- Daily application caps per consultant are set by the owner. The hub stops assigning once the cap is reached.

3.4 Answer bank and approval workflow

- The answer bank stores reusable answers to screening questions, per consultant (for example: years of experience with a tool, notice period, willingness to relocate).
- When the consultant app hits a question it cannot answer, the question is sent to the hub and appears on that consultant's dashboard as an unknown.
- The consultant types the answer. It sits as pending. The recruiter assigned to that consultant approves or corrects it. Only then does it enter the answer bank and get used on future applications.
- Exception: any answer touching salary expectations or work authorization routes to the owner for approval, regardless of recruiter.
- No answer is ever auto-approved. A consultant can never approve their own answer.

3.5 Record store

The hub stores, permanently, for every application:

- The application record: consultant, job, company, portal, date and time, and final status (submitted, waiting on consultant, stalled on login, or skipped with reason).
- The exact resume PDF that was submitted, as a file, linked to the record.
- Every question the form asked and the exact answer filled in, as a question-and-answer list attached to the record.
- A full audit log: every login, every resume download, every answer approval, every assignment change, every token issued or revoked — with who, what, when, and from which machine.

3.6 Contact discovery engine

Runs immediately after job matching, for every job that enters a queue. It follows a four-step waterfall, stopping at the first step that produces a usable contact:

1. **Read the posting.** LinkedIn and Dice postings usually name the poster; Greenhouse and Lever postings usually do not. If a personal name is found, carry it to step 2.
2. **Check the contact store first.** If that person (or, when no poster was named, that company and location) already has a contact pulled within the last 90 days, reuse it. Zero cost. This is the default path once the store fills up.
3. **Enrich through the licensed provider.** If the contact is missing or older than 90 days, look the person up by name and company through the Apollo API to get business email, then use Lusha or SignalHire credits to obtain the phone number. Store the result with source and date.
4. **Fallback for empty or generic posters.** If no personal name exists (posting says “HR Team” or nothing), search the provider for two active talent-acquisition or recruiting people at that company in that work location, ranked by seniority match to the role, and enrich both. Both attach to the job.
 - Every job in a queue carries its contact(s): name, title, company, business email, phone, and source. The consultant sees them on that queue item; the recruiter and owner see them everywhere.
 - De-duplication is by person name plus company. One person contacted for five jobs is one record linked five times, not five pulls.
 - Each contact record has a do-not-contact flag. Anyone who asks not to be contacted is flagged once by a recruiter and the system stops attaching them, permanently.
 - Hard rule: the system never scrapes LinkedIn profiles itself. All contact data comes through the licensed providers' official interfaces. LinkedIn Sales Navigator remains a manual tool for recruiters and is not connected to the system, because it has no official interface for this.

4. Web Portal — Roles and Permissions

Three access levels. Every user has exactly one role. Sessions require a password plus a one-time code (two-step verification) for Owner and Recruiter accounts.

Capability	Owner	Recruiter	Consultant
Add or remove recruiters; assign consultants to recruiters	Yes	No	No
See all consultants, all applications, all records	Yes	No	No
See assigned consultants' dashboards, queues, records	Yes	Yes (own list only)	No
Approve pending answers	Yes (all)	Yes (own consultants; except salary and work authorization)	No

Capability	Owner	Recruiter	Consultant
Approve salary and work-authorization answers	Yes (only)	No	No
Set daily application caps	Yes	No	No
Issue and revoke consultant app tokens	Yes	No	No
View own application history, resumes, queue status	Yes (all)	Yes (own consultants)	Yes (self only)
Fill answers for unknown questions	No	No	Yes (self only, pending approval)
Edit search criteria for a consultant	Yes	Yes (own consultants)	No
View audit log	Yes	No	No
View contacts attached to a job	Yes	Yes (own consultants' jobs)	Yes (own queue items only)
Search the full contact store; flag do-not-contact	Yes	Yes	No
Export contacts in bulk	Yes (only)	No	No

4.1 Dashboard views

- **Owner:** agency-wide numbers (applications per day and week, per recruiter, per consultant), overlap flags, stalled logins across the bench, pending salary and work-authorization approvals, and a search box that returns every application ever made for a name or company, with the resume and answers attached.
- **Recruiter:** the same views limited to their assigned consultants, plus their approval inbox for pending answers.
- **Consultant:** their own application history, which resume went out for each job, current queue status, their unknowns list to fill in, and the contact card (name, title, email, phone) on each job in their queue. Every contact view is logged.

5. Consultant Desktop App

5.1 Installation and identity

- A small desktop program installed once per consultant machine (Windows and Mac).

- On first run it activates with a one-time token issued by the owner from the portal. The token binds to that person and that machine. The owner can revoke it from the dashboard at any time, which kills the app instantly.

5.2 Portal logins — how authentication works

- The app never holds or transmits any portal password. On first use of each portal (LinkedIn, Dice, a Workday company site, and so on) the app opens a real browser window and steps aside. The consultant logs in themselves, including any code sent to their phone.
- The browser's logged-in session is saved in a protected folder on the consultant's machine only — the same way a normal browser stays signed in. Later runs reuse the saved session; no password entry needed.
- When a session expires, the app detects it, pauses that portal's queue, notifies the consultant to log in again, and reports the stall to the hub so it shows on the recruiter and owner dashboards.
- Corporate portals such as Workday require one account per company. The consultant creates each once; the session is saved the same way. Expect more one-time logins in the first two weeks, then it settles.

5.3 The application cycle

1. Every 4 hours (with a small random offset so activity does not look machine-timed), the app wakes and pulls the consultant's queue from the hub.
2. For each queue item it opens the portal in the real browser with the saved session, navigates to the job, and fills every form field it can from the hub-provided data and approved answers. It attaches the tailored resume.
3. Unknown questions are sent to the hub as unknowns and that application is parked until the answer is approved.
4. Completed forms stop at the review screen. The app lists them for the consultant, who reviews and clicks submit on each. The app confirms submission and reports it to the hub with the full question-and-answer list.
5. The app deletes local working files after each cycle. Nothing permanent stays on the machine except the saved browser sessions and the app itself.

5.4 Behavior limits built into the app

- Human-paced form filling: realistic typing speed and pauses, one application at a time, never parallel.
- The daily cap set by the owner is enforced locally as well as at the hub.
- LinkedIn gets the most conservative treatment: lowest volume, human submit always, and the app stops touching LinkedIn for the day if the portal shows any bot-check challenge.

6. Security and Misuse Controls

- Per-person, per-machine tokens, revocable instantly from the owner dashboard.
- Consultants get read-only access to their own profile and queue. They cannot see other candidates, edit criteria, or change answers directly.
- Resumes are delivered per job, never in bulk. Every delivery is logged with time, person, and machine.
- Two-person rule on every reusable answer: one fills, a different person approves.
- Daily application caps per consultant, set centrally.
- All hub data encrypted at rest; all traffic between app, portal, and hub encrypted in transit (standard HTTPS/TLS).
- Hub server hardened basics: firewall allowing only web traffic, automatic security updates, daily encrypted backups of the database and resume files kept off the server, and a restore test once a month.
- The AI no-fabrication rule and the second-check comparison live only in hub code; no portal user at any level can weaken them.

7. Data Model (plain-language)

The database holds these record types. The developer will turn this into tables; the names below are the required content, not final column names.

Record	What it holds
User	Name, email, role (owner / recruiter / consultant), login credentials, two-step setting, active or disabled.
Assignment	Which consultant belongs to which recruiter, with history of changes (who moved whom, when).
Consultant profile	Contact details, work-authorization status, base resume file, search criteria, daily cap.
Job posting	Source, company, title, location, work type, pay if listed, full description text, fingerprint for de-duplication, first-seen time.
Queue item	Consultant, job, tailored resume file link, status (queued / filled / parked-unknown / awaiting-submit / submitted / skipped), timestamps.
Application record	Everything in Section 3.5, permanent.
Answer	Consultant, question text, answer text, status (pending / approved / rejected), who approved, when; flag for salary / work-authorization routing.

Record	What it holds
Contact	Person name, title, company, work location, business email, phone, data source, date pulled, do-not-contact flag. De-duplicated by name plus company; re-pull allowed only after 90 days.
Contact link	Which contact is attached to which job posting, and when. One contact can link to many jobs.
Device token	Consultant, machine fingerprint, issued date, revoked date if any.
Audit event	Actor, action, target, timestamp, machine. Append-only; nothing in this table can be edited or deleted.

8. AI Model Selection and Running Cost

The system uses AI in three places. All three run on Anthropic's Claude models through their pay-per-use interface (the Claude API). Prices below are the published rates as of July 2026, charged per million tokens — a token is roughly three-quarters of a word, so a million tokens is about 750,000 words processed. Input is what you send in; output is what the model writes back.

8.1 Recommended model per stage

Stage	Model	Price per 1M tokens (in / out)	Why this model
Job matching and work-type classification (hub, high volume)	Claude Haiku 4.5, sent through the Batch discount	\$1 / \$5 standard; \$0.50 / \$2.50 with the 50% batch discount	This is a simple yes/no matching task done thousands of times a day. Haiku is Anthropic's cheapest current model and is more than capable of it. The 4-hour cycle is not time-sensitive, so the half-price batch mode (results within an hour or so) fits perfectly.
Resume tailoring (hub, quality-critical)	Claude Sonnet 4.6, with prompt caching on the base resume and rules	\$3 / \$15; the cached portion of input costs 90% less on repeat calls	This is the step where quality earns money — a well-tailored resume gets interviews. Sonnet follows the keep-the-style and no-fabrication rules reliably; the cheaper Haiku produces sloppier edits on this task, and the flagship Opus (\$5 / \$25) adds

Stage	Model	Price per 1M tokens (in / out)	Why this model
			cost without a visible gain here. Caching pays off because the same base resume and rule set are resent for every job.
Fabrication check on each tailored resume (hub)	Claude Haiku 4.5	\$1 / \$5	Comparing two resume versions and flagging unsupported claims is a simple checking task. Using a second, different call as the checker is cheap insurance on the no-fabrication rule.
Unknown-question matching (consultant app, live)	Claude Haiku 4.5	\$1 / \$5	Deciding whether a new form question already matches an answer in the bank (“years with SQL” vs “how long have you used SQL”) is simple and needs a fast response while the form is open. Haiku is the fastest and cheapest fit.
Development itself (build phase)	Claude Code running Sonnet 4.6 (its default); switch to Opus 4.8 only for a stubborn bug	Covered by the developer's Claude subscription (Section 10)	Sonnet 4.6 is the standard production coding model and what Claude Code uses by default. Paying Opus rates for everyday coding is wasted money; keep it for the rare hard problem.

Upgrade path: if pilot feedback says tailored resumes need to be stronger, switch the tailoring stage to Claude Opus 4.8 (\$5 / \$25). That is a one-line configuration change, not a rebuild. Do not start there; test whether Sonnet is enough first — it usually is.

Cheaper models exist elsewhere (for example Google's Gemini Flash tier or budget models from other vendors, some priced under \$1 per million tokens). They were considered and set aside for two reasons: the resume-tailoring step depends on precise instruction-following where Claude is strong, and running everything on one vendor means one bill, one account to secure, and the same vendor the developer already uses through Claude Code. The savings from mixing vendors here would be under \$50 a month — not worth the added complexity.

8.2 Expected monthly AI cost

Working assumptions: 20 active consultants, about 8 matched jobs per consultant per day, keyword pre-filter in place so the AI only sees plausible postings.

Stage	Estimated monthly cost	Notes
Job matching (Haiku, batch)	\$20 – \$50	Depends on how tight the keyword pre-filter is. Without the pre-filter this stage alone would run \$150+; the pre-filter is required.
Resume tailoring (Sonnet, cached)	\$120 – \$180	About 160 tailored resumes a day. The largest AI line item, and the one that earns interviews.
Fabrication check (Haiku)	\$10 – \$20	One short check per tailored resume.
Unknown-question matching (Haiku)	Under \$10	Volume drops every week as the answer bank fills.
Total at 20 consultants	\$160 – \$260 per month	Scales roughly \$8 – \$13 per active consultant per month. At 50 consultants, plan on \$400 – \$650.

These are usage-based costs with no minimum commitment. Set a monthly spending cap in the Anthropic billing console (start at \$400) so a bug in a loop can never run up a surprise bill.

8.3 Contact-data cost (separate from AI)

Contact discovery spends provider credits, not AI tokens. Phone numbers cost several times more credits than emails on every provider. The 90-day reuse rule is the main cost control: month one is the expensive month while the store fills; from month two onward most jobs hit the store and cost nothing. The developer must pull current credit pricing from the owner's existing Apollo, Lusha, and SignalHire plans before build and confirm the monthly allowance covers roughly 1,500 to 2,500 new-contact pulls in month one at 20 consultants, dropping to a few hundred per month after. The hub must track credit spend per provider on the owner dashboard and stop pulling (reuse-only mode) when a monthly credit budget set by the owner is reached.

9. Technology Stack

Chosen for one criterion: a junior developer working with Claude Code can build and maintain every part in one language, with the most widely documented tools.

Part	Technology	Reason
Hub service and 4-hour scheduler	Python with FastAPI; scheduled jobs via the hub's built-in scheduler (APScheduler) or system cron	Python is the most Claude-Code-friendly language for a junior developer, and every library needed (job feeds, PDF handling, AI calls) is mature in Python.
Database	PostgreSQL	The standard reliable choice. Handles the record store and audit log for years without attention.
Resume file storage	S3-compatible object storage (AWS S3 or equivalent), encrypted	Cheap, durable, and keeps large files out of the database.
Web portal	React front end served by the same FastAPI hub	Standard pairing; Claude Code generates and maintains both sides well.
Consultant desktop app	Python with Playwright (browser automation) using a persistent browser profile per portal, packaged into an installer with PyInstaller; small system-tray interface	Playwright is the current standard for driving a real browser and supports saved login sessions natively. Same language as the hub, so the junior developer maintains one skill set.
Hub hosting	One small cloud server (for example a \$20–\$40/month virtual server at AWS, Hetzner, or DigitalOcean) plus managed PostgreSQL	This workload is light on the server side; the heavy browser work happens on consultant machines.
Contact data	Apollo API as primary (people search plus business email); Lusha and SignalHire APIs for phone-number fallback. LinkedIn Sales Navigator stays manual-only.	The owner already pays for these. Licensed providers deliver the data through official channels, which keeps the system off LinkedIn scraping and the account bans that come with it.
AI access	Anthropic Claude API (Section 8)	One vendor for both the product AI and the development tool.

10. Build Plan — Junior Developer with Claude Code

Claude Code is Anthropic's development tool: it runs in a terminal or editor on the developer's machine, reads the project, writes the code across files, runs it, and fixes its own errors under the developer's

direction. A junior developer working this way should deliver the pilot version in 11 to 13 weeks. The steps below are the working instructions to hand them along with this document.

10.1 One-time setup (day one)

1. Install Git, Node.js 18 or newer (needed to install Claude Code), and Python 3.11 or newer.
2. Install Claude Code: open a terminal and run `npm install -g @anthropic-ai/claude-code` — then run `claude` and sign in.
3. Subscription: put the developer on a Claude Max plan (\$100/month tier) for the build period. It covers heavy daily Claude Code use at a flat rate, which is cheaper than pay-per-token at build-phase volumes. Drop to a lower plan after launch, when usage becomes occasional maintenance.
4. Create a private GitHub repository named `staffing-autopilot` with three folders: `hub/` (server, scheduler, portal back end), `portal/` (React front end), and `app/` (consultant desktop app), plus `docs/` containing this specification saved as `SPEC.md` (a plain-text copy).
5. Create a file named `CLAUDE.md` in the repository root. Claude Code reads this file automatically at the start of every session. It must contain: the technology stack from Section 9; the rule that portal passwords are never stored or transmitted; the no-fabrication rule text for the tailoring prompt; the rule that the audit table is append-only; and the instruction to write tests for every feature.

10.2 Working method (every day)

- Work one phase at a time (Section 10.3). Start each Claude Code session by pointing it at the relevant spec section, for example: “Read docs/SPEC.md Sections 3 and 7. Build the database models and a FastAPI service in `hub/` covering users, assignments, consultant profiles, job postings, queue items, applications, answers, device tokens, and the append-only audit table. Include a seed script with sample data and tests.”
- Review every change Claude Code proposes before accepting it. Run the tests. Never accept code you have not read.
- Commit to Git after every working feature — never let more than half a day of work sit uncommitted. Work on a branch; merge to main only when tests pass.
- When stuck for more than 30 minutes, paste the full error into Claude Code and ask it to explain the cause before asking for a fix. If a session has gone in circles, start a fresh session and restate the goal in one clear paragraph.
- Never put real candidate data into the system until Phase 8 security review is done. Build and test with the seeded fake data.

10.3 Build phases and order

Phase	Weeks	Deliverable
1. Database and hub core	1–2	All data models from Section 7, the FastAPI service, seed data, tests passing.
2. Portal with three roles	2–4	Login with two-step verification, the permission matrix from Section 4 enforced server-side, owner / recruiter / consultant dashboards with seeded data.
3. Answer bank and approval flow	4–5	Unknowns appear on consultant dashboards; consultant fills, recruiter approves; salary and work-authorization routing to owner; two-person rule enforced.
4. Job discovery	5–6	Greenhouse and Lever public feeds ingested, keyword pre-filter, de-duplication, Haiku batch matching, queues populating on the 4-hour cycle.
5. Resume tailoring	6–7	Sonnet tailoring with the locked prompt, Haiku fabrication check, PDF output stored and linked, flagged resumes routed to recruiter.
6. Contact discovery	7–8	The Section 3.6 waterfall: poster extraction, contact store with 90-day reuse, Apollo email enrichment, Lusha/SignalHire phone fallback, two-recruiter fallback search, do-not-contact flag, credit budget cutoff, contact cards on queue items.
7. Consultant app	8–10	Token activation, saved-login flow, queue pull, Playwright form fillers for Greenhouse and Lever, stop-at-review-screen, reporting back, local cleanup, installer built.
8. More portals	10–12	Dice form filler, then Workday. LinkedIn last, with the conservative limits from Section 5.4.
9. Security review and pilot prep	12–13	Token revocation tested end to end, backups and restore tested, spending caps set (AI and contact credits), audit log verified append-only, then pilot (Section 11).

Portal form fillers are the part most likely to need ongoing maintenance: portals change their forms without notice. Budget roughly one developer-day per month after launch for filler repairs, and build each filler so a page-layout failure parks the application with a clear error instead of submitting bad data.

11. Pilot Rollout Plan

1. Pick one recruiter and three bench consultants in one job category. Load their profiles, base resumes, and criteria.
2. Run two weeks on Greenhouse and Lever portals only, daily cap of 5 applications per consultant.
3. Owner and recruiter review every tailored resume and every submitted application in week one. Loosen to spot-checks in week two if quality holds.
4. Success gate before wider rollout: zero fabricated resume content found, at least 90 percent of queued applications reaching the review screen without manual fixes, unknowns per consultant falling week over week, no portal account warnings, and contact accuracy above 80 percent on a recruiter spot-check of 25 pulled contacts (email deliverable, phone reaches the right company).
5. Then add Dice, raise caps, and onboard the bench in groups of five consultants per week so support load stays manageable.

12. Assumptions and Open Items

- Consultants consent in writing to the agency processing their resumes and application data through this system. A one-page consent form should be signed at onboarding; have your attorney review it.
- Automated interaction sits against the terms of service of some portals even with a human submit. The design minimizes risk (human pace, human submit, low caps, consultant-owned logins), but a portal can still restrict an account. Consultants should be told this plainly in the consent form.
- Prices in Section 8 are the published rates as of July 2026 and can change; the developer should confirm current rates at claude.com/pricing before launch.
- Contact data is business contact information (work email, work phone, title) used for legitimate recruiting outreach. The system stores no personal home details, honors the do-not-contact flag permanently, and pulls only through licensed providers whose terms permit this use. The consultant consent form should state that the contact store is agency property.
- Owner decisions locked in this version: indefinite record retention; consultant-fills-recruiter-approves answer flow; salary and work-authorization answers to owner; independent consultant queues; human final submit; contacts visible to consultants per queue item only, full store searchable by owner and recruiters.